

A Python Mini Project
On
Student Record Management System

Submitted By

Sudheera Mewan
Registration No: CIT-25-02-0325
Module: Programing Concept

Submitted to

Ms. Nil Upuli Ekanayake
Module Owner



Faculty of Computing & IT
Sri Lanka Technological Campus

03-2026

Abstract

- This student management system is a desktop application create using python **Tkinter**.
- This system allows to manage personal information such as student id, name, email, age, gender, mobile number, address and batch.
- The application the practical implementation of core Python concepts, including data structures, file handling and GUI programming.
- Through this system, users have the ability to create ,read, modify, and delete student records.
- The data entered into this system is obtained in csv file
- Key achievements include the successful implementation of data validation for age and email formats, the prevention of duplicate student IDs, and real-time table updates. The application serves as an excellent demonstration of object-oriented programming concepts combined with procedural programming in Python

Table Of Content	
Introduction	1
• Background	1.1
• Objective	1.2
System Overview	2
• System Architecture	2.1
• System Components	2.2
• System Components	2.3
User Interface (Tkinter)	2.3.1
Data Management Module	2.3.2
File Handling Module	2.3.3
System Workflow	2.3.4
Technologies Used	3
• Development Environment	3.1
• Python Libraries	3.2
Implementation Details	4
• Project description	4.1

- Features of the system - User Interface Features - Data Management Features - Error Handling Features	4.2 4.2.1 4.2.2 4.2.3
- Data structures are used - Set - Dictionary - List - Tuple	4.3 4.3.1 4.3.2 4.3.3 4.3.4
- File handling - Loading Data - Saving Data	4.4 4.4.1 4.4.2
Screenshots of GUI	4.5
Python Concepts - Basic Python Concepts - Control Structures - Input Validation - List of User-Defined Functions	5 5.1 5.2 5.3 5.4
Testing - Test Cases - Test Case 1: Add Student - Test Case 2: Update Student - Test Case 3: Delete Student - Test Case 4: Search Functionality - Test Case 5: File Handling - Edge Cases Tested	6 6.1 6.1.1 6.1.2 6.1.3 6.1.4 6.1.5 6.2
Future Improvements - Short-term Enhancements - Long-term Improvements - Database Integration - Enhanced Features - Technical Improvements - New Modules	7 7.1 7.2 7.2.1 7.2.2 7.2.3 7.2.4
Conclusion - References - Tools Used	8 8.1 8.2

1.INTRODUCTION

1.1 Background

In educational institutions, managing student records efficiently is crucial for administrative tasks. Traditional paper-based systems are time-consuming and prone to errors. Digital record management systems offer a reliable alternative, enabling quick access, easy updates, and secure storage of student information.

1.2 Objective

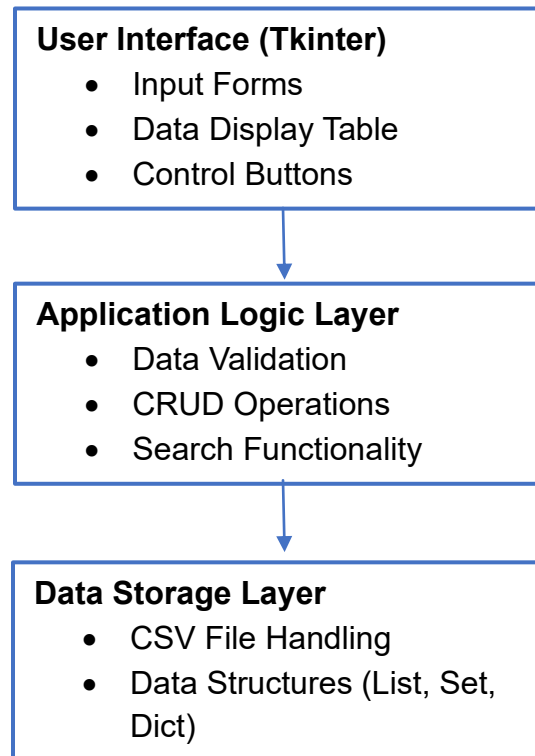
The primary objective of this project is to develop a user-friendly Student Record Management System that

- Provides a graphical interface for easy data entry
- Ensures data integrity through validation
- Prevents duplicate records
- Enables quick search and retrieval
- Maintains persistent storage using files
- Demonstrates practical use of Python data structures

2. SYSTEM OVERVIEW

2.1 System Architecture

The application follows a simple yet effective architecture: -



2.2 System Components

2.2.1 User Interface (Tkinter)

Main window with organized frames

- Input form for data entry
- Search panel for record retrieval
- Treeview table for data display
- Button controls for operations

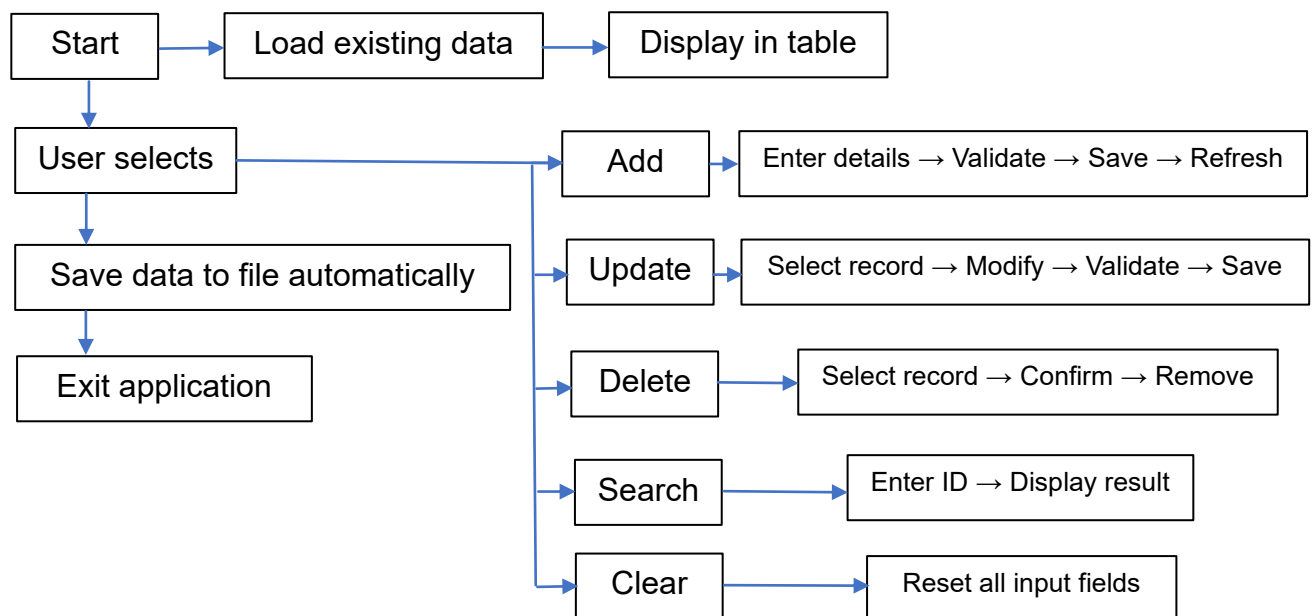
2.2.2 Data Management Module

- Add new student records
- Update existing records
- Delete records
- Search by Student ID
- Display all records

2.2.3 File Handling Module

- Read data from CSV file
- Write data to CSV file
- Handle file operations safely
- Error handling for file operations

2.3 System Workflow



3. TOOLS AND TECHNOLOGIES USED

3.1 Development Environment

Programming Language – Python

GUI Framework – Tkinter

IDE – Pycharm

Operation System – Ubuntu

3.2 Python Libraries

Tkinter - Designing the GUI

Ttk - Themed Tkinter widgets

CSV - File Handling and Store the Data

MessageBox - Dialog boxes for user notifications

os - File system operations

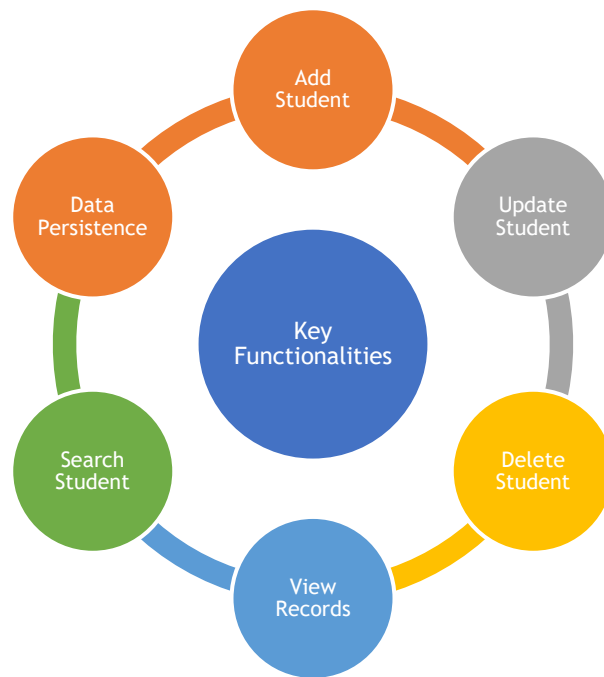
```
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3 import csv
4 import os
```

```
student_ids = set()
```

4. IMPLEMENTATION DETAILS

4.1 Project Description

The Student Record Management System is a desktop application that provides a complete interface for managing student information. The project demonstrates the practical application of Python programming concepts in building a real-world solution.



- Add Student - Insert new student records with validation
- View Records - Display all students in a tabular format
- Update Student - Modify existing student information
- Delete Student - Remove student records permanently
- Search Student - Find specific student by ID
- Data Persistence - Save/load data from CSV file

4.2 Features of the System

4.2.1 User Interface Features

- Clean Layout - Organized frames for different functionalities
- Input Validation - Real-time data validation with error messages
- Responsive Design - Scrollable table for large datasets
- Color Coding - Visual distinction for different operations
- Keyboard Friendly - Tab navigation between fields

4.2.2 Data Management Features

- Duplicate Prevention - Set data structure ensures unique IDs
- Data Validation - Age range checking, email format validation
- Automatic Saving - Data saved after every operation
- Search Functionality - Quick lookup by Student ID
- Bulk Display - Show all records with scroll support

4.2.3 Error Handling Features

- Input Error Messages - Clear guidance for invalid inputs
- File Operation Errors - Graceful handling of file issues
- Confirmation Dialogs - Prevent accidental deletions
- Empty Field Checks - Ensure all required data is entered

4.3 Data Structures Are Used

4.3.1 Set

Python -

```
student_ids = set()
```

Usage –

```
if s_id in student_ids:  
    student_ids.add(s_id)  
    student_ids.remove(s_id)
```

- Purpose - Maintains unique Student IDs with lookup time
- Benefit - Prevents duplicate entries efficiently

4.3.2 Dictionary

Python -

```
student_records = {}
```

Structure –

```
student_records[s_id] = {  
    "name": name,  
    "email": email,  
    "age": age,  
    "gender": gender,  
    "Mobile Number": mobile,  
    "Address": address,  
    "Batch": batch,  
}  
student_list.append(row)
```

- Purpose - Maps Student IDs to complete student information
- Benefit - Provides fast access to student details using ID as key

4.3.3 List

Python –

```
student_list = []
```

Usage –

```
student_list.append(row)
```

- Purpose - Maintains ordered collection of all records
- Benefit - Preserves insertion order and allows sequential processing

4.3.4 Tuple

Python –

```
HEADERS = ("Student ID", "Student Name", "Email", "Age", "Gender",  
           "Mobile Number", "Address", "Batch") # Tuple with 8 fields
```

- Purpose - Immutable sequence of column headers
- Benefit - Ensures headers remain constant throughout the program

4.4 File Handling

4.4.1 Loading Data

Python -

```
def load_data():  
    if not os.path.exists(FILE_NAME):  
        return  
    try:  
        with open(FILE_NAME, mode='r', newline='') as file:  
            reader = csv.reader(file)  
            next(reader, None) # Skip header  
            for row in reader:  
                if len(row) == 8: # Updated to 8 fields  
                    s_id, name, email, age, gender, mobile, address, batch = row  
                    student_ids.add(s_id)  
                    student_records[s_id] = {  
                        "name": name,  
                        "email": email,  
                        "age": age,  
                        "gender": gender,  
                        "Mobile Number": mobile,  
                        "Address": address,  
                        "Batch": batch,  
                    }  
                    student_list.append(row)  
    except Exception as e:  
        messagebox.showerror("File Error", f"Could not load data: {e}")
```

Process -

- Check if file exists
- Open file in read mode
- Skip header row
- Read each row and populate data structures
- Handle exceptions gracefully

4.4.2 Saving Data

Python –

```
def save_data():
    try:
        with open(FILE_NAME, mode='w', newline='') as file:
            writer = csv.writer(file)
            writer.writerow(HEADERS)
            for s_id, details in student_records.items():
                writer.writerow([
                    s_id,
                    details["name"],
                    details["email"],
                    details["age"],
                    details["gender"],
                    details["Mobile Number"],
                    details["Address"],
                    details["Batch"]
                ])
    except Exception as e:
        messagebox.showerror("File Error", f"Could not save data: {e}")
```

Process -

- Open file in write mode
- Write headers first
- Iterate through dictionary and write each record
- Handle any file operation errors

4.5 Screenshots of GUI

Main Window (Figure 1)

Search by ID:

Student Information

Student ID: Mobile Number:
Student Name: Address:
Email: Batch:
Age:
Gender:

Student Records

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
------------	--------------	-------	-----	--------	---------------	---------	-------

Figure 1

Add Student Operation

New student adds

Step 01

File the all-student information details. (All fields are required)

Search by ID:

Student Information

Student ID: Mobile Number:
Student Name: Address:
Email: Batch:
Age:
Gender:

Student Records

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
------------	--------------	-------	-----	--------	---------------	---------	-------

Step 02

Step 01

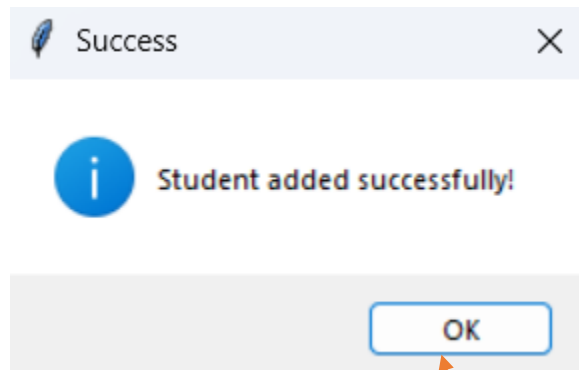
Step 02

After Click the add button

Display message

Step 03

Click OK



Step 03

After click OK button and display entry details in Student Records Display

Student Records							
Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0325	sudheera mewar	cit-25-02-0325@gmail.com	23	male	0777549425	125/B katugasthota mithotalulla	2029B

Search Functionality

Step 01

Enter student ID search area

Step 01

Student Record Management System

Search

Search by ID:

Student Information

Student ID:
Student Name:
Email:
Age:
Gender:
Mobile Number:
Address:
Batch:

Student Records

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0325	sudheera mewar	cit-25-02-0325@gmail.com	23	male	0777549425	125/B katugasthota mithotalulla	2029B
cit-25-02-0001	alice	cit-25-02-0001@gmail.com	20	male	0710032158	14/A badulla	2029B
cit-25-02-0002	charli	cit-25-02-0002@gmail.com	21	male	0777857528	247/A mathara	2029B
cit-25-02-0003	Bob	cit-25-02-0003@gmail.com	23	male	0725849765	157/B meepe road waga	2028A

Step 02

After click search Button

Result –

Search Result

Student Record Management System

Search

Search by ID:

Student Information

Student ID:
Student Name:
Email:
Age:
Gender:
Mobile Number:
Address:
Batch:

Student Records

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0001	alice	cit-25-02-0001@gmail.com	20	male	0710032158	14/A badulla	2029B

Update Functionality

Step 01

Find student id in Student Records Display

The screenshot shows the 'Student Record Management System' window. At the top, there is a 'Search' section with a text input field containing 'cit-25-02-0001', a 'Search' button, and a 'Show All' button. Below this is the 'Student Information' section with input fields for Student ID, Student Name, Email, Age, Gender, Mobile Number, Address, and Batch. At the bottom, there is a 'Student Records' table with columns: Student ID, Student Name, Email, Age, Gender, Mobile Number, Address, and Batch. The table contains one row with the following data: Student ID: cit-25-02-0001, Student Name: alice, Email: cit-25-02-0001@gmail.com, Age: 20, Gender: male, Mobile Number: 0710032158, Address: 14/A badulla, Batch: 2029B.

Step 02(Figure 2)

Click the student details row

The screenshot shows the 'Student Record Management System' window. The 'Search' section has the same input field and buttons. The 'Student Information' section now has pre-filled values: Student ID: cit-25-02-0001, Student Name: alice, Email: cit-25-02-0001@gmail.com, Age: 20, Gender: male, Mobile Number: 710032158, Address: 14/A badulla, and Batch: 2029B. The 'Student Records' table is the same as in the previous screenshot. An orange arrow points to the first row of the table, which is highlighted in blue. Below the table, the text 'Click this' is written.

Figure 2

Automaticity fille the student information details

Now you can change previous details

Student Record Management System

Search

Search by ID:

Student Information

Student ID: Mobile Number:

Student Name: Address:

Email: Batch:

Age:

Gender:

Student Records

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0001	alice	cit-25-02-0001@gmail.com	20	male	0710032158	14/A badulla	2029B

After change click update button

Figure 3

After change student details and click Update button. (Figure 3)

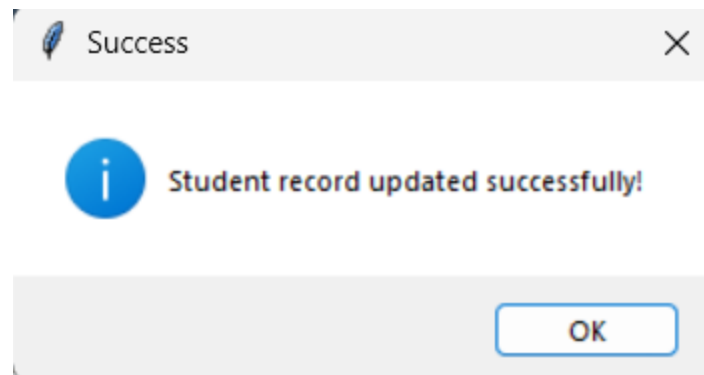


Figure 4 (Success message)

Output of update –

Success full the update student details

The screenshot shows the 'Student Record Management System' window. At the top, there is a 'Search' section with a 'Search by ID' field containing 'cit-25-02-0001', a 'Search' button, and a 'Show All' button. Below this is the 'Student Information' section with fields for Student ID, Student Name, Email, Age, Gender, Mobile Number, Address, and Batch. The 'Update' button is highlighted in orange. At the bottom, the 'Student Records' table is displayed with columns: Student ID, Student Name, Email, Age, Gender, Mobile Number, Address, and Batch. The table contains five rows of student data. The row for 'cit-25-02-0001' (Alice) has its 'Email' and 'Address' cells highlighted with orange borders.

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0325	sudheera mewan	cit-25-02-0325@gmail.com	23	male	0777549425	125/B katugasthota mithotalulla	2029B
cit-25-02-0001	alice	cit-25-02-0001@sltc.edu.lk	20	male	710032158	14/A sri jayawradanapura kotta	2029B
cit-25-02-0002	charli	cit-25-02-0002@gmail.com	21	male	0777857528	247/A mathara	2029B
cit-25-02-0003	Bob	cit-25-02-0003@gmail.com	23	male	0725849765	157/B meepe road waga	2028A

Delete Student Recode -

Step 01

Select student details in the student records display. (Figure 5)

The screenshot shows the 'Student Record Management System' window. The 'Delete' button in the 'Student Information' section is highlighted with a red box. An orange arrow points from the 'Delete' button to the 'Student Name' column of the 'Student Records' table. The table contains five rows of student data. The row for 'cit-25-02-0001' (Alice) is highlighted in blue. The text 'Step 02' is written in the bottom right corner of the window.

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0325	sudheera mewan	cit-25-02-0325@gmail.com	23	male	0777549425	125/B katugasthota mithotalulla	2029B
cit-25-02-0001	alice	cit-25-02-0001@sltc.edu.lk	20	male	710032158	14/A sri jayawradanapura kotta	2029B
cit-25-02-0002	charli	cit-25-02-0002@gmail.com	21	male	0777857528	247/A mathara	2029B
cit-25-02-0003	Bob	cit-25-02-0003@gmail.com	23	male	0725849765	157/B meepe road waga	2028A

Figure 5

Step 02 –

Now click **delete** button.

Step 03 –

Now display the confirmation. (*Figure 6*)

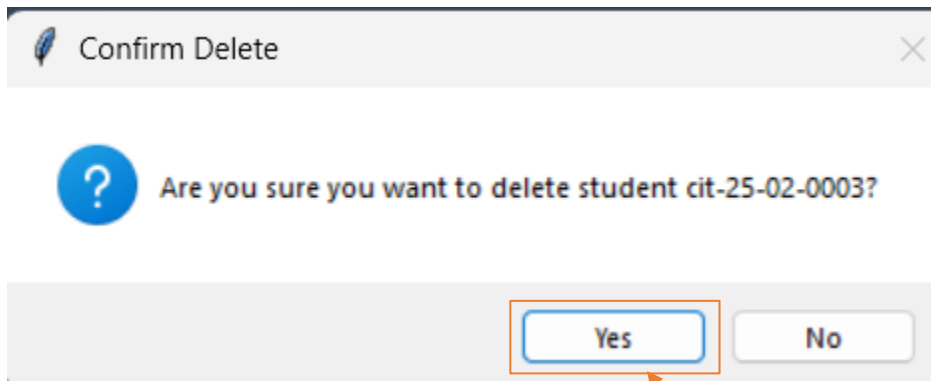


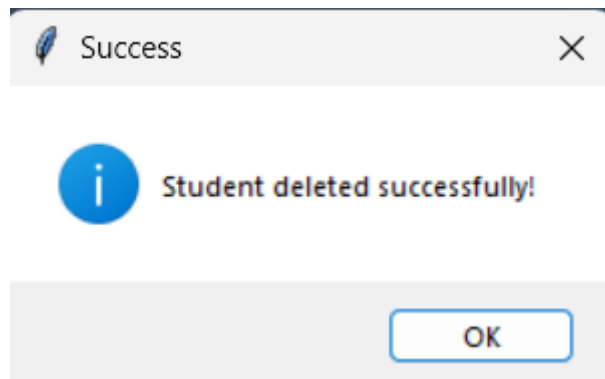
Figure 6

Step 04

Step 04 -

After Click the **Yes** Button

Success Message



After click OK Button

Output of Delete –

Student Record Management System

— □ ×

Search

Search by ID:

Search

Show All

Student Information

Student ID:

Student Name:

Email:

Age:

Gender:

Mobile Number:

Address:

Batch:

Add

Update

Delete

Clear

Exit

Student Records

Student ID	Student Name	Email	Age	Gender	Mobile Number	Address	Batch
cit-25-02-0325	sudheera mewan	cit-25-02-0325@gmail.com	23	male	0777549425	125/B katugasthota mithotalulla	2029B
cit-25-02-0001	alice	cit-25-02-0001@sltc.edu.lk	20	male	710032158	14/A sri jayawradanapura kotta	2029B
cit-25-02-0002	charli	cit-25-02-0002@gmail.com	21	male	0777857528	247/A mathara	2029B

5. USE OF PYTHON CONCEPTS

5.1 Basic Python Concepts

Concept	Implementation
Variables	<code>USERNAME</code> , <code>PASSWORD</code> , <code>FILE_NAME</code> constants
Functions	15+ user-defined functions including <code>add_student()</code> , <code>update_student()</code> , etc.
Classes	<code>StudentManagementSystem</code> class with 20+ methods
Exception Handling	try-except blocks for file operations and data conversion

5.2 Control Structures

If-Else Conditions

```
if search_id in student_records:
    # Clear table
    for item in tree.get_children():
        tree.delete(item)
    # Insert only the searched student
    details = student_records[search_id]
    tree.insert("", "end", values=(
        search_id,
        details["name"],
        details["email"],
        details["age"],
        details["gender"],
        details["Mobile Number"],
        details["Address"],
        details["Batch"]
    ))
else:
    messagebox.showinfo("Not Found", f"No student found with ID: {search_id}")
```

For Loops

```
for i, col in enumerate(columns):  
    tree.heading(col, text=col)  
    tree.column(col, width=column_widths[i], anchor=tk.CENTER)
```

5.3 Input Validation

```
for i, col in enumerate(columns):  
    tree.heading(col, text=col)  
    tree.column(col, width=column_widths[i], anchor=tk.CENTER)
```

5.4 List of User-Defined Functions

- `load_data()` - Loads data from CSV file
- `save_data()` - Saves data to CSV file
- `refresh_table()` - Updates the display table
- `clear_fields()` - Clears all input fields
- `add_student()` - Adds new student record
- `on_tree_select()` - Handles table row selection
- `update_student()` - Updates existing record
- `delete_student()` - Deletes selected record
- `search_student()` - Searches by Student ID
- `show_all()` - Displays all records
- `setup_gui()` - Sets up the GUI layout
- `login()` - Handles login authentication
- `run()` - Starts the main application loop
- `validate_age()` - Validates age input
- `validate_marks()` - Validates marks input

6. TESTING

6.1 Test Cases

6.1.1 Test Case 1: Add Student

Test	Input	Expected Result	Actual Result	Status
Valid Data	ID: cit-25-02-0325, Name:Sudheera mewan, Age: 20,Gender: Male, Mobile number: 0777549425, Address: 195/B pinnawala waga, Batch: 2029B	Record added successfully	Record added to table and CSV	Pass
Duplicate ID	cit-25-02-0325 (existing)	Error message	"Student ID already exists!"	Pass
Invalid Age	100	Error message	"Age must be between 1-75"	Pass
Invalid Email Format	Cit-25-02-0325sltc.edu.lk	Error message	"Please enter valid email"	pass
Empty Fields	Missing name	Warning message	"All fields are required!"	pass

6.1.2 Test Case 2: Update Student

Test	Input	Expected Result	Actual Result	Status
Update Existing	Select record, modify name	Record updated	Changes reflected in table and CSV	Pass
No Selection	Click Update without selection	Error message	"Please select a student"	Pass
Update with Invalid Data	Update with age: -5	Error message	Validation error displayed	Pass

6.1.3 Test Case 3: Delete Student

Test	Input	Expected Result	Actual Result	Status
Delete Existing	Select record, confirm	Record removed	Record deleted from table and CSV	Pass
Cancel Delete	Select record, cancel	No deletion	Record remains	Pass
No Selection	Click Delete without selection	Error message	"Please select a student"	Pass

6.1.4 Test Case 4: Search Functionality

Test	Input	Expected Result	Actual Result	Status
Search Existing	Cit-25-02-0001	Record displayed	Only matching record shown	Pass
Search Non-existing	Cit-25-02-500	Not found message	"No student found"	Pass
Empty Search	Click Search without input	Warning	"Please enter Student ID"	Pass
Show All	Click after search	All records displayed	Full table restored	Pass

6.1.5 Test Case 5: File Handling

Test	Input	Expected Result	Actual Result	Status
Save on Add	Add new record	CSV file updated	File contains new record	Pass
Save on Update	Update record	CSV file updated	Changes reflected in file	Pass
Save on Delete	Delete record	CSV file updated	Record removed from file	Pass
Load on Start	Open application	Previous records loaded	All records displayed	Pass
File Missing	Delete CSV, start app	Empty table, no error	Empty table, new file created on save	Pass

6.2 Edge Cases Tested

- Empty input fields
- Negative age values
- Invalid email formats
- Duplicate Student IDs
- Special characters in names
- Extremely long inputs
- Rapid clicking of buttons
- File permission errors
- Missing data file

7. FUTURE IMPROVEMENTS

7.1 Short-term Enhancements

Enhancement	Description	Priority
Export to Excel	Generate Excel reports	High
Print Functionality	Print student records	Medium
Batch Import	Import multiple students from CSV	High
Data Backup	Automatic backup before save	Low

7.2 Long-term Improvements

7.2.1 Database Integration

- Migrate from CSV to MySQL/SQLite
- Support for multiple concurrent users
- Better data integrity and relationships

7.2.2 Enhanced Features

Advanced Search

- Search by name, Batch, Mobile Number
- Filter by gender, age, Location group
- Sort by any column

Analytics Dashboard

- Class performance statistics
- Display student image
- Graphical charts (matplotlib integration)

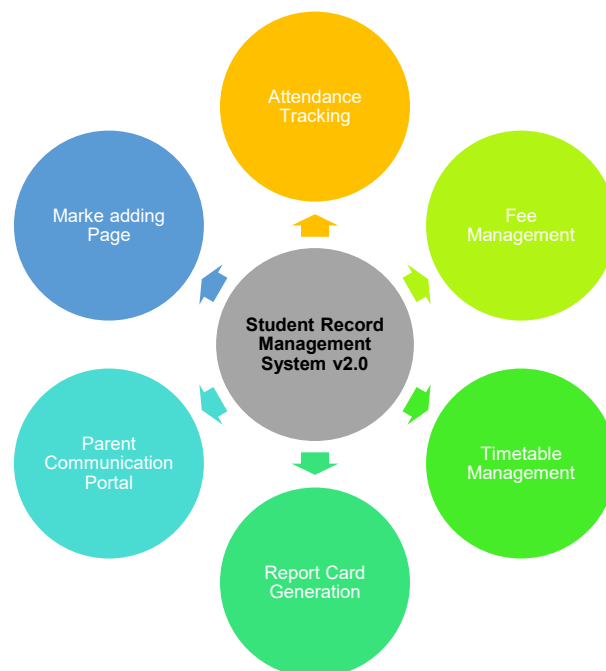
User Management

- Multiple user roles (Admin, Teacher, Staff)
- Activity logging
- Access control levels

7.2.3 Technical Improvements

Area	Current	Proposed
Data Storage	CSV file	SQLite/MySQL database
UI Framework	Tkinter	PyQt or CustomTkinter
Error Handling	Basic try-except	Comprehensive logging
Testing	Manual	Unit tests with pytest
Deployment	Source code	Executable (.exe)

7.2.4 New Modules



8. Conclusion

It has been a great pleasure for me to work on this exciting and challenging project. This project proved good for me as it provided practical knowledge of not only programming in Python based applications.

It also provides knowledge about the latest technology used in development.e. This will provide better opportunities and guidance in future in developing projects independently.

8.1 References

Online Resources

1. Python Official Documentation
 - <https://docs.python.org/3/library/tkinter.html>
 - <https://docs.python.org/3/library/csv.html>
2. Tkinter Tutorials
 - <https://realpython.com/python-gui-tkinter/>
 - https://www.tutorialspoint.com/python/python_gui_programming.htm
3. CSV File Handling
 - <https://www.geeksforgeeks.org/working-csv-files-python/>
4. Video Tutorial
 - <https://youtu.be/flz0Cm5Cb4U?si=HgUtptKwrJYoTTMP>

Books

- "Python Programming: An Introduction to Computer Science" by John Zelle
- "Tkinter GUI Application Development Blueprints" by Bhaskar Chaudhary

8.2 Tools Used

- Python 3.x: <https://www.python.org/downloads/>
- Visual Studio Code: <https://code.visualstudio.com/>
- Git: <https://git-scm.com/>